Group 1

Personal Information Management System Design Document v1

1 Introduction

This document describes the software design of the command-line Personal Information Management system specified in the SRS. The system manages notes, tasks, events, and contacts, and supports creation, modification, search, display, deletion, saving, and loading. This document explains the MVC architecture, the main software components and their interfaces, and the collaboration between components during a search-and-update scenario.

2 System Architecture

The system adopts the Model-View-Controller architectural pattern to separate terminal interaction from record-management logic. The view reads commands and displays responses, while the controller interprets commands and coordinates model operations. The model contains record definitions, validation, collection management, search criteria, and persistence.

This separation allows model operations to be tested without interactive terminal input and keeps terminal-specific code outside the required model package. In this implementation, the controller also assembles response messages and converts records into display strings, while the view handles terminal input and output.

Group 1

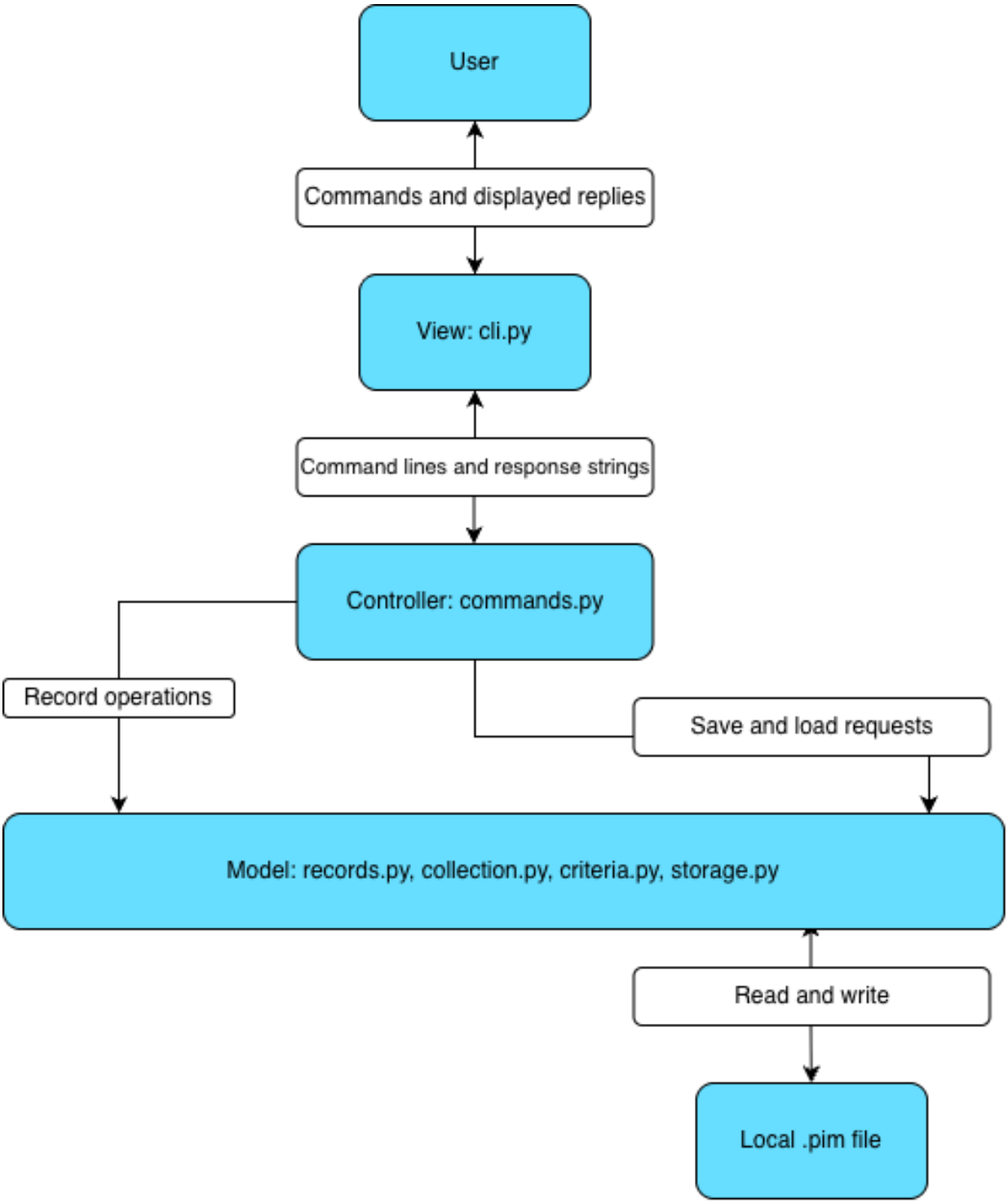


Figure 1. MVC architecture of the PIM system

Arrows from the controller to the model indicate operation requests; results return to the controller

Group 1

Part	Actual file/module	Responsibility
View	view/cli.py	Read terminal input, display replies, manage the session loop
Controller	controller/commands.py	Interpret commands, call model operations, assemble responses, handle expected errors
Model	model/records.py	Define records and validate their values
Model	model/collection.py	Manage the active collection of records
Model	model/criteria.py	Parse and evaluate search conditions
Model	model/storage.py	Save and load .pim files

3 Main Components and Class Design

3.1 Record Classes

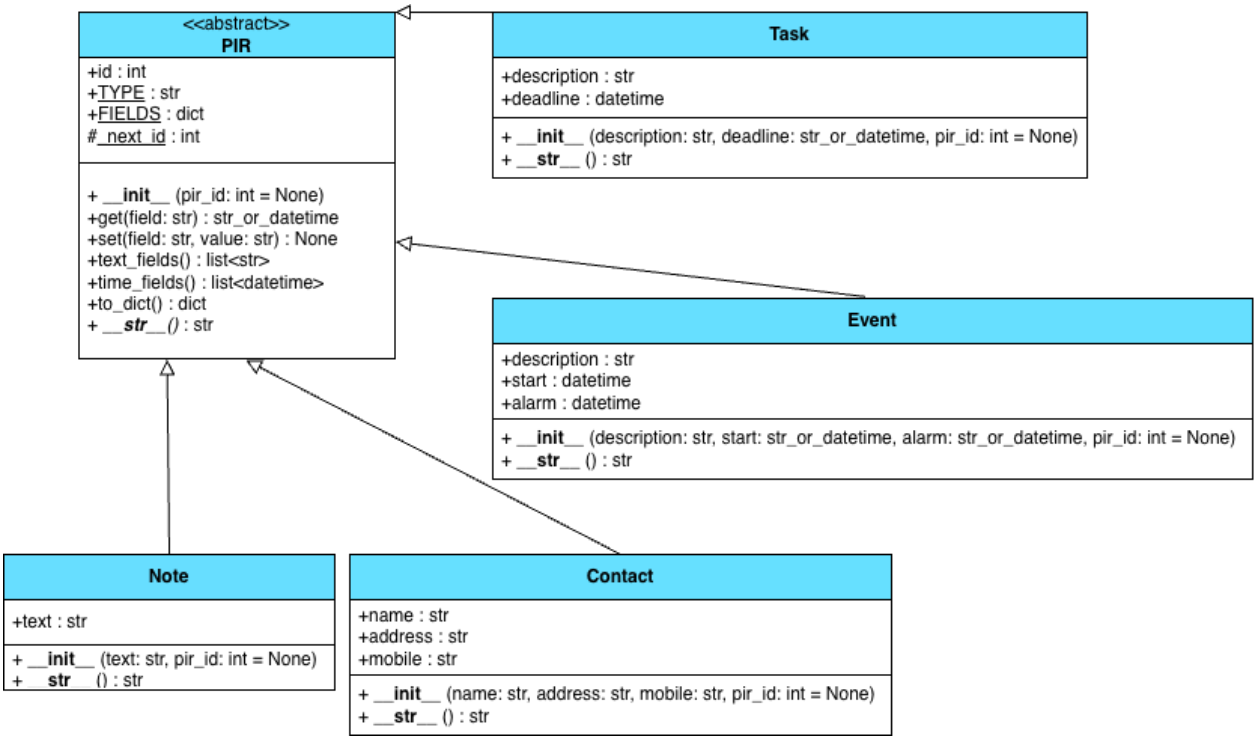


Figure 2. Record class hierarchy of the PIM system

Group 1

PIR is the abstract base class for all personal information records. It provides a common ID, field access, field modification, and conversion to a dictionary. Note, Task, Event, and Contact inherit these operations and define their own fields and string representations. Each subclass defines TYPE to identify its record type and FIELDS to classify its fields as text or time values. This metadata allows modification, searching, and serialization to handle different record types consistently. Text values are validated before assignment, and time values are converted to datetime objects.

Class	**Attributes**	**Purpose**
PIR	id: int	Identifies a record.
PIR	TYPE: str, FIELDS: dict	Class-level metadata describing the record type and its fields.
PIR	_next_id: int	Shared class-level counter used to allocate IDs.
Note	text: str	Stores note content.
Task	description: str, deadline: datetime	Stores a task and its deadline.
Event	description: str, start: datetime, alarm: datetime	Stores event details and its start and alarm timestamps.
Contact	name: str, address: str, mobile: str	Stores contact details, preserving phone-number formatting.

Group 1

Method	**Parameters**	**Returns**	**Expected exceptions**	**Purpose**
PIR.__init__	pir_id: int | None = None	None	None for valid inputs	Assign an ID and advance the shared counter when necessary.
PIR.get	field: str	str | datetime	KeyError	Retrieve a supported field value.
PIR.set	field: str, value: str	None	KeyError, ValueError	Validate and replace a supported field value.
PIR.text_fields	None	list[str]	None for valid records	Return the record's text values for searching.
PIR.time_fields	None	list[datetime]	None for valid records	Return the record's time values for searching.
PIR.to_dict	None	dict	None for valid records	Produce record data for saving, formatting time values as strings.
__str__	None	str	None for valid records	Return the record's ID, type, and stored fields as display text.

All four constructors return None and may raise ValueError when required text or time values are invalid

Group 1

Constructor	Parameters	Purpose
Note.__init__	text: str, pir_id: int | None = None	Validate and initialize a note.
Task.__init__	description: str, deadline: str | datetime, pir_id: int | None = None	Validate the description and convert the deadline to a datetime.
Event.__init__	description: str, start: str | datetime, alarm: str | datetime, pir_id: int | None = None	Validate the description and convert the start and alarm times.
Contact.__init__	name: str, address: str, mobile: str, pir_id: int | None = None	Validate and initialize contact details.

Types not explicitly annotated in the source code are inferred from the implementation. Python does not declare checked exceptions; the tables list expected exceptions during normal use.

3.2 Collection and Controller

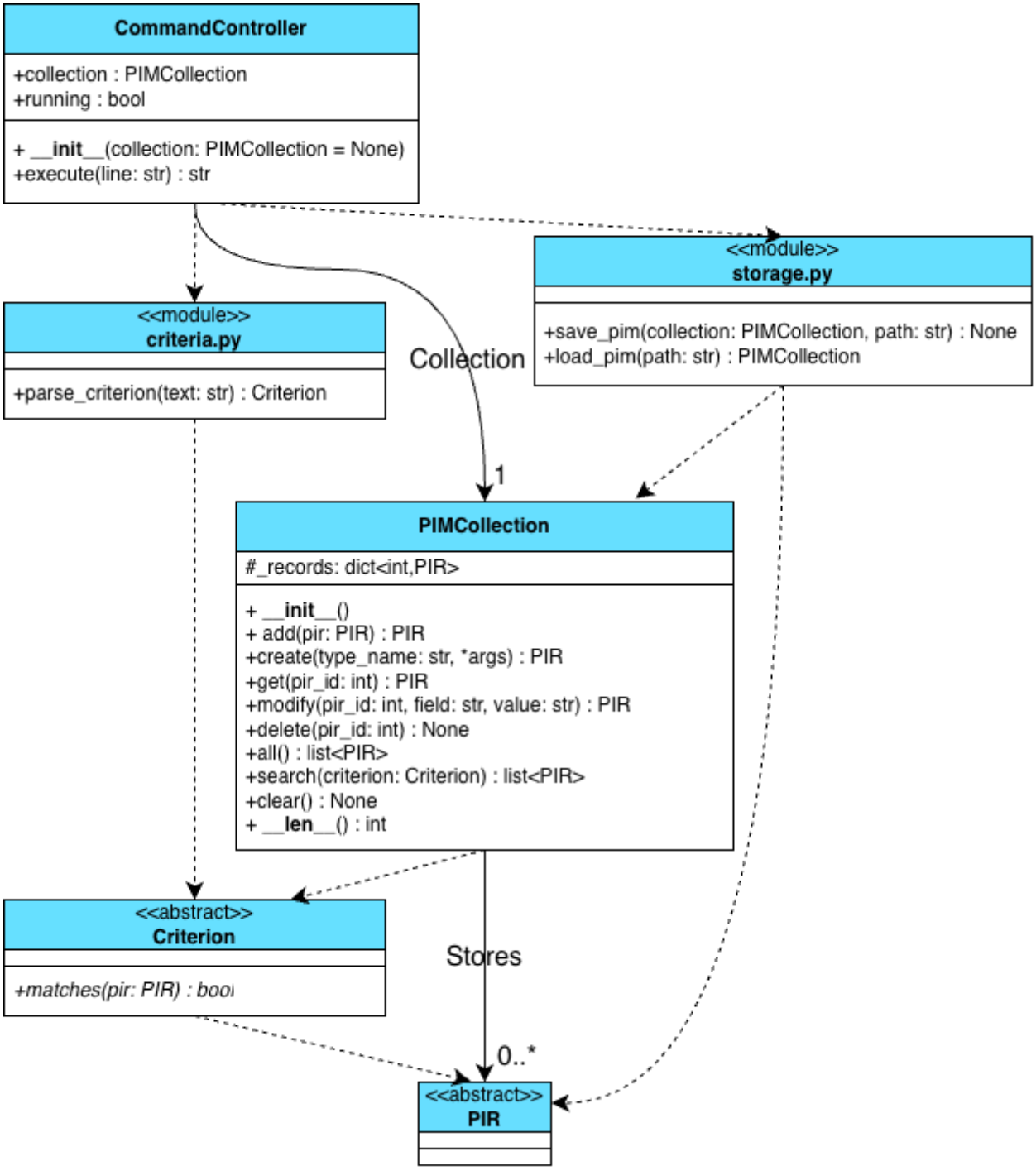


Figure 3. Main components and dependencies of the PIM system.

Group 1

PIMCollection manages the active records using the _records dictionary which maps integer IDs to PIR objects. It provides creation, retrieval, modification, deletion, and searching operations. Modification delegates field validation to PIR.set(), while searching calls matches() on the supplied criterion for each record. Results are returned in ascending ID order.

CommandController holds the active collection and a Boolean running flag. Its execute() method identifies the command and dispatches it to the corresponding command handler. The controller converts expected ValueError, KeyError, and OSError exceptions into error messages for the view. A successful load replaces its collection reference, while the quit command sets running to False.

Group 1

Method	**Parameters**	**Returns**	**Expected exceptions**	**Purpose**
__init__	None	None	None	Initialize an empty record dictionary.
add	pir: PIR	PIR	None for valid records	Store a record using its ID and return it.
create	type_name: str, *args: record constructor arguments	PIR	ValueError; TypeError for incorrect constructor arguments	Construct and store the selected record type.
get	pir_id: int	PIR	KeyError	Retrieve a record by ID.
modify	pir_id: int, field: str, value: str	PIR	KeyError, ValueError	Validate and update one field, then return the record.
delete	pir_id: int	None	KeyError	Remove the specified record.
all	None	list[PIR]	None for valid records	Return all records in ascending ID order.
search	criterion: Criterion	list[PIR]	None for valid criteria and records	Return records satisfying the criterion in ascending ID order.
clear	None	None	None	Remove all records from the collection.
__len__	None	int	None	Return the number of stored records.

Group 1

Method	Parameters	Returns	Expected exceptions	Purpose
CommandController.__init__	collection: PIMCollection | None = None	None	None for valid inputs	Keep the supplied collection or create an empty one; initialize running to True.
CommandController.execute	line: str	str	Handles ValueError, KeyError, and OSError internally	Process a command and return response text. Empty input returns an empty string.

The view/cli.py module provides run_cli(controller: CommandController | None = None) -> None. It reads commands while the controller's running flag is True, passes each command to execute(), and displays non-empty responses. EOFError and KeyboardInterrupt are handled to end the session.

3.3 Search Criteria

The search mechanism uses the abstract Criterion class, which defines matches(pir: PIR) -> bool. Each concrete criterion checks whether a record satisfies a particular condition. PIMCollection.search() applies the criterion to each record and returns matching records in ascending ID order.

TypeCriterion filters records by type. ContainsCriterion performs case-insensitive substring matching across text fields. TimeCriterion compares the record's time fields with a specified datetime using <, >, or =. AndCriterion, OrCriterion, and NotCriterion combine criteria to support more complex searches.

Class	**Main attributes**	**Constructor parameters**	**Matching behaviour**
TypeCriterion	type_name: str	type_name: str	Checks the record type.
ContainsCriterion	needle: str	needle: str	Checks whether any text field contains the search text, ignoring case.
TimeCriterion	op: str; when: datetime	op: str; when: str or datetime	Checks whether any time field satisfies the comparison.
AndCriterion	left: Criterion; right: Criterion	left: Criterion; right: Criterion	Returns True when both criteria match.
OrCriterion	left: Criterion; right: Criterion	left: Criterion; right: Criterion	Returns True when either criterion matches.
NotCriterion	inner: Criterion	inner: Criterion	Returns the opposite of the inner criterion's result.

All six concrete classes inherit from Criterion and implement matches (pir: PIR) -> bool. Their constructors return None. AndCriterion and OrCriterion each hold references to two Criterion objects, while NotCriterion holds a reference to one. These relationships allow nested search expressions. TimeCriterion raises ValueError when an unsupported comparison operator or an invalid datetime string is supplied.

The criteria.py module provides parse_criterion(text: str) -> Criterion. It converts a search expression into a criterion object or a combination of criterion objects. The parser supports parentheses and applies operator precedence in the order NOT (!), AND (&&), then OR (||). Empty or invalid expressions raise ValueError, which CommandController.execute() catches and converts into an error message.

3.4 Persistence

The storage.py module handles saving and loading records using local .pim files. Each record is stored as one JSON object per line, including its type, ID, and field values. Datetime values use the format YYYY-MM-DD HH:MM. This format allows records to be reconstructed with their original types and IDs.

The storage functions are separate from the command controller and record classes. CommandController calls save_pim() to save the current collection and load_pim() to create a collection from a file. The controller replaces its active collection only after loading completes successfully. If loading fails, the existing active collection remains available.

Function	Parameters	Return type	Exceptions	purpose
save_pim	collection: PIMCollection; path: str	None	ValueError; OSError	Saves all records in ascending ID order, replacing the file's existing contents.
load_pim	path: str	PIMCollection	ValueError; OSError	Reads and validates records, then returns a new collection.
_check_ext	path: str	None	ValueError	Checks that the file extension is .pim, ignoring case.

During loading, blank lines are ignored. Each remaining line must contain a valid JSON record with a supported record type, a unique positive integer ID, and valid field values. Invalid records cause ValueError with line information. File access failures raise OSError. CommandController.execute() catches these exceptions and returns an error message to the user.

4 Search and Update Interaction

Precondition: The active collection contains a task with ID 3. The user first enters search type=task. After identifying Task 3 in the search results, the user enters modify 3 deadline “2026-11-15 18:00”.

Postcondition: Task 3 has the new deadline. Its ID, type, description, and all other records remain unchanged.

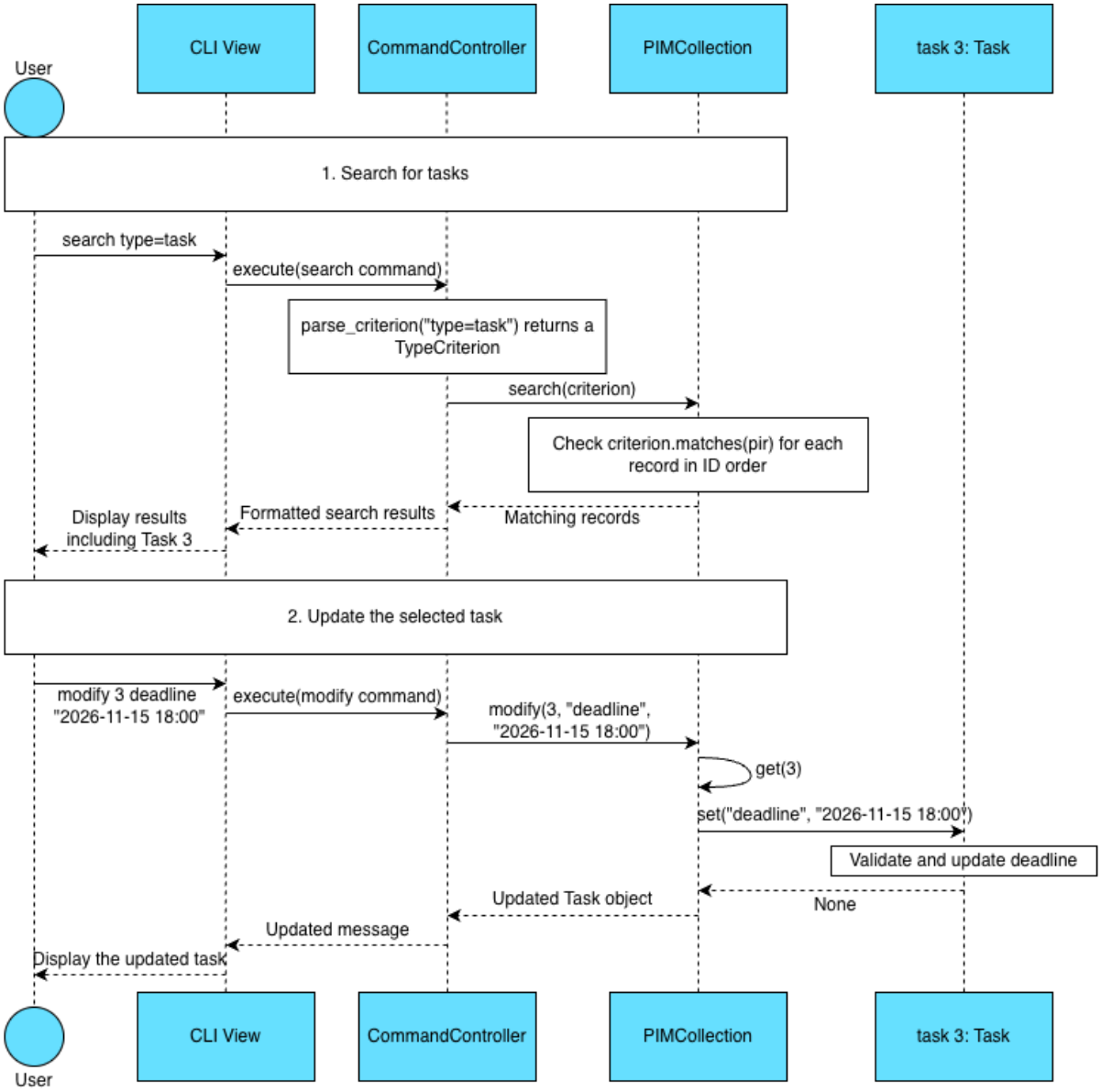


Figure 4. Sequence of searching for tasks and updating a selected task

The controller parses the search criterion and asks the collection for matching records. After the CLI displays the results, the user selects Task 3 by ID and enters a separate command to change its deadline. The collection retrieves the task and calls its set() method to validate and apply the new deadline. The CLI then displays the update confirmation.

5 Design Decisions and Requirements Traceability

Design choice	**Reason**	**Related user stories**
PIR Inheritance	Reuse common field access, validation, and serialization behaviour	US1-US6
Criterion hierarchy	Evaluate simple and combined search conditions through a common interface	US7
ID-based collection	Select a specific record for modification, display, or deletion	US6, US8, US9
Validate before updating	Preserve existing values when replacement input is invalid	US6
Load into a new collection before replacement	Preserve active records if loading fails	US11
JSON Lines persistence	Store and restore record types, IDs, and fields	US10-US11